

Kai Ruchi

A complete online shop for a homemade South Indian food business — and a plain-English explanation of every part of it.

கை ருசி · கೈ ராசி

kai ruchi — "hand-taste". The South Indian word for why the same recipe tastes different depending on whose hands made it.

FRONTEND

Next.js 16

BACKEND

FastAPI

DATABASE

PostgreSQL

MIGRATIONS

Alembic

REPOSITORY

github.com/niri2630/kai-ruchi

01 What you have actually built

Kai Ruchi is a working online shop. Not a design, not a mock-up — a real one. You can browse products, search them, put them in a basket, make an account, pay, and then watch your order move from "placed" to "delivered".

It sells the food a single home kitchen in Udupi makes: masalas, pickles, fresh batters, fried snacks and sweets. Fourteen products across five categories.

What a visitor can do

- ◆ **Browse** — a home page, five category pages, and a page for every product.
- ◆ **Search** — type a word and matching products appear instantly as you type.
- ◆ **Filter and sort** — by category, by how spicy it is, vegetarian or not, in stock or not, and by price or rating.
- ◆ **Add to a cart** — which works even before you log in, and follows you into your account when you do log in.
- ◆ **Make an account** — sign up, sign in, edit your details, save delivery addresses.
- ◆ **Check out and pay** — enter a delivery address and pay (on a simulated payment screen — see section 8).
- ◆ **Track the order** — a six-step timeline showing exactly where the parcel is.
- ◆ **Write reviews** — and reviews are marked "verified" automatically if that account really bought the product.
- ◆ **Contact the shop** — a contact form that saves the message into the database.

SAY THIS OUT LOUD

The one-sentence description

"It is a full-stack e-commerce web application. The frontend is built with Next.js and React, the backend is a REST API built with FastAPI in Python, and the data is stored in a PostgreSQL database with Alembic managing the schema changes."

What it deliberately does *not* have

There is **no admin dashboard**. That was a scope decision — it is a mini project, not a business. Two consequences follow, and it is worth knowing them because a teacher may well ask:

- **Orders move on a timer.** A real shop has staff clicking "packed" and "shipped". Here, once an order is paid, it walks itself along the tracking timeline over about 35 minutes so it can be demonstrated live.
- **Products are added by editing a seed file** (backend/app/seed.py) and re-running it, rather than through an admin screen.

02 The words you need to know

Learn these ten and you can hold any conversation about this project. Everything else is detail.

WORD	WHAT IT ACTUALLY MEANS
Frontend	Everything the user sees and clicks — the pages, buttons, images, animations. It runs inside the visitor's browser.
Backend	The program running on a server that the frontend talks to. It does the thinking: checking passwords, calculating totals, saving orders. Nobody sees it.
Database	Where information is permanently stored — users, products, orders. If the server restarts, the database still remembers everything.
API	The list of things the frontend is allowed to ask the backend for. Like a restaurant menu: "give me all products", "add this to my cart".
Endpoint	One single item on that menu. For example <code>GET /api/products</code> means "give me the products".
REST	The common convention for designing those menus, using web addresses plus verbs — GET (fetch), POST (create), PATCH (change), DELETE (remove).
JSON	The text format the frontend and backend use to send information to each other. Looks like <code>{"name": "Mango Pickle", "price": 279}</code> .
Schema	The shape of the database — which tables exist and what columns they have.
Migration	A recorded change to that shape. Instead of editing the database by hand, you write a migration, so the change can be repeated on any computer.
Deploy	Putting the project on the internet so anyone can visit it, instead of only on your own laptop.

Two more that sound scary

JWT (JSON Web Token)

When you log in, the backend gives your browser a long signed string. Your browser sends it with every later request, which is how the server knows it is still you without asking for your password again.

ORM (Object-Relational Mapper)

A translator between Python code and database tables. Instead of writing SQL by hand, you write `db.query(Product)` and the ORM — here, SQLAlchemy — writes the SQL for you.

03 The three pieces, and how they talk

Every full-stack project is the same three boxes. Once you can draw this on a whiteboard, you understand the architecture.

BROWSER (the visitor's phone or laptop)

Next.js frontend – pages, buttons, animations

▼ asks for data over HTTPS, in JSON

FASTAPI BACKEND (a Python program on a server)

checks passwords • calculates totals • applies the rules

▼ reads and writes rows via SQLAlchemy

POSTGRESQL DATABASE

users • products • carts • orders • reviews

Why keep them separate?

Because they have different jobs and different risks. The frontend can be rewritten without touching the data. The backend enforces rules the browser cannot be trusted with — a visitor could edit the page in their browser and set a price to ₹1, but the backend recalculates the real price before saving the order, so it does not matter.

SAY THIS OUT LOUD

"The frontend never trusts its own numbers. When you check out, the browser sends only the product IDs and quantities. The backend looks the real prices up in the database and recalculates the total itself. That way the price cannot be tampered with."

A real example: what happens when you click "Add to cart"

1. You click the green + button on a product card.
2. The frontend sends `POST /api/cart/items` with `{product_id: 5, quantity: 1}`.
3. The backend checks: **does this product exist? is it still in stock?** If there are only 3 left and you asked for 4, it refuses and sends back the message "Only 3 left in this batch."
4. If you are logged in, it finds the cart attached to your account. If not, it uses a random token stored in your browser, so guests get a cart too.
5. It saves the row, then recalculates the subtotal, the delivery fee (free over ₹799), and the total.
6. It sends the whole updated cart back as JSON.
7. The frontend updates the cart badge in the corner, pops the number, and shows a small toast saying "Mango Pickle added".

04 Every technology, and why it was chosen

A teacher will almost certainly ask "why did you use X?" Here is a real answer for each one, not just a description.

Frontend

TECHNOLOGY	WHAT IT DOES	WHY THIS ONE
Next.js 16	The React framework that gives us routing, page structure and the build system.	Every folder inside <code>src/app</code> automatically becomes a page — no routing to configure. It also deploys to Vercel with no setup.
React 19	Builds the interface out of reusable components.	A product card is written once and used on six different pages.
TypeScript	JavaScript with type checking.	If the backend returns a price as text and the code treats it as a number, the error is caught while writing, not by a user.
Tailwind CSS v4	Styling written as small class names directly on the element.	No separate stylesheet to keep in sync, and the design tokens (colours, fonts) live in one file.
Framer Motion	All the animation.	It can animate elements as they enter the screen and as they leave it — plain CSS cannot animate something that is being removed.
Lenis	Smooth scrolling.	Makes the page glide instead of jumping line by line, and lets other animations follow the scroll position smoothly.
Zustand	Holds shared state — who is logged in, what is in the cart.	The navbar and the cart page both need the cart. Zustand keeps one copy that both read from.

Backend

TECHNOLOGY	WHAT IT DOES	WHY THIS ONE
FastAPI	The Python web framework that provides the API.	It generates interactive API documentation automatically — open <code>/docs</code> and every endpoint is listed and testable. Excellent to show a teacher.
SQLAlchemy 2	The ORM — talks to the database using Python objects.	Lets us write Python instead of raw SQL, and prevents SQL-injection attacks by design.
Pydantic v2	Validates every piece of incoming data.	If someone posts a review with a rating of 9, it is rejected before it ever reaches the database.
Alembic	Manages database migrations.	The schema history is recorded in files, so the exact same database can be rebuilt on any computer with one command.
bcrypt	Hashes passwords.	Passwords are never stored. Only a one-way scramble is saved, so even with the database, nobody can read them.
python-jose	Creates and verifies the JWT login tokens.	Signed with a secret key, so a token cannot be forged.

Database

TECHNOLOGY	WHAT IT DOES	WHY THIS ONE
PostgreSQL	Stores all the data.	Free, extremely reliable, and supports proper relationships between tables, exact decimal money values, and enum types for order status.
Supabase	Hosts that PostgreSQL database online, free.	Needed because a database on your laptop is not reachable from the internet.

05 How the database is designed

Nine tables. Every one exists for a reason, and the reasons are the interesting part.

TABLE	HOLDS
users	Name, email, phone, and the hashed password.
addresses	Saved delivery addresses. One user can have many.
categories	The five shelves — masalas, pickles, fresh batters, snacks, sweets.
products	The fourteen items: name, price, stock, spice level, description, ingredients, photo.
carts	One open cart per user — or per guest, keyed by a random browser token.
cart_items	One row per product in a cart, with its quantity.
orders	A confirmed purchase: totals, payment status, and the delivery address.
order_items	What was actually in that order.
order_events	One row each time the order changes status. This is what the tracking timeline reads.
reviews	Star rating and text. One review per person per product.
contact_messages	Messages sent through the contact form.

Three design decisions worth explaining

SAY THIS OUT LOUD

1. Why `order_items` copies the product name and price

It looks like duplicated data, and it is deliberate. If the price of Mango Pickle changes next month, a customer's old invoice must still show what they actually paid. So the order stores its own copy. The same goes for the delivery address — editing your saved address must not rewrite where a past parcel was sent. This is called **snapshotting**.

SAY THIS OUT LOUD

2. Why there is a separate `order_events` table

The order could just have a "status" column. But then you would only know where it is *now*, not when it was packed or when it shipped. A separate table with one row per change gives a full history, which is exactly what the tracking timeline displays.

SAY THIS OUT LOUD

3. Why guests get a cart too

Forcing people to sign up before they can add anything to a basket loses sales. So a guest cart is stored against a random token in the browser. When that person later signs in, the backend merges the guest cart into their account cart instead of throwing it away.

Relationships, in one line each

- ◆ A **category** has many **products**. A product belongs to one category.
- ◆ A **user** has many **addresses**, many **orders** and many **reviews**.
- ◆ A **cart** has many **cart_items**; each points at one product.
- ◆ An **order** has many **order_items** and many **order_events**.
- ◆ A **review** joins one user to one product — and the database enforces that a person can only review a product once.

06 The API, endpoint by endpoint

Thirty-six endpoints, grouped into nine files. You do not need to memorise them — you need to know the pattern.

METHOD	ADDRESS	WHAT IT DOES
POST	/api/auth/signup	Create an account, return a login token
POST	/api/auth/login	Sign in, return a login token
GET	/api/auth/me	Who am I? (requires the token)
GET	/api/categories	All five shelves, with product counts
GET	/api/products	Products, with filtering, sorting and paging
GET	/api/products/suggest	Type-ahead results for the search box
GET	/api/products/{slug}	Everything about one product
GET	/api/cart	My cart, with totals
POST	/api/cart/items	Add something to the cart
PATCH	/api/cart/items/{id}	Change a quantity
DELETE	/api/cart/items/{id}	Remove a line
POST	/api/orders/checkout	Turn the cart into an order
POST	/api/payments/verify	Confirm the payment
GET	/api/orders/{number}	One order plus its tracking timeline
POST	/api/reviews/product/{slug}	Write a review
POST	/api/contact	Send a message

DEMO TRICK

With the backend running, open **<http://127.0.0.1:8000/docs>** in a browser. FastAPI generates a live, clickable page listing every endpoint, which you can run right there. Showing this to a teacher is far more convincing than a slide.

How the folders map to the API

```
backend/app/  
  models/      the database tables, as Python classes  
  schemas/     the shape of data going in and out (validation)  
  routers/     the endpoints themselves, one file per topic  
  services/    the business rules – cart totals, order timeline, payments  
  core/        settings, database connection, security, shared helpers  
  seed.py      the starting catalogue: 14 products, 6 demo users, 16 reviews
```

07 How the design was decided

If asked "why does it look like this?", the answer is not "it looks nice". Every choice comes from the subject.

The colours

Taken off a South Indian kitchen shelf rather than picked from a palette tool:

NAME	COLOUR	WHERE IT COMES FROM
Chilli	#e23e2e	Byadgi chilli, and the kumkum at a doorway
Turmeric	#f5a31a	Fresh-ground turmeric
Banana leaf	#1e7a54	The plate a South Indian meal is served on
Indigo	#4b2e83	Temple indigo
Jasmine	#fff6e9	The page background — mallige flowers
Lamp soot	#241209	All the text

The three signature ideas

Glassmorphism

Cards are frosted glass because *a pickle jar is glass* — you shop by looking through it. Not a trend pick; it comes from the product.

Claymorphism

Buttons are soft and puffy with a hard bottom edge that compresses when pressed, so they feel like objects you push.

The masala mesh

Five spice-coloured blobs drift slowly behind the whole site. Every frosted surface is showing you that, blurred.

Kolam

The rice-flour pattern drawn on a South Indian doorstep every morning. Used as a motif — and because it is a real dot grid, it earns its place.

The typefaces

- ◆ **Bricolage Grotesque** — all headings. Modern and slightly irregular.
- ◆ **Instrument Sans** — body text. Quiet and very readable.
- ◆ **Baloo Thambi 2** — the "KaiRuchi" wordmark. Chosen because it was designed alongside Tamil script, so the brand name sits in a face built for its own language.

The animation, and its limits

Lenis drives all scrolling. On top of it: a loading screen that draws a kolam, words that rise into place, section reveals as you scroll, magnetic buttons, cards that tilt in 3D with a moving highlight, and letter-by-letter flips on the menu.

SAY THIS OUT LOUD

"Every animation respects the operating system's **prefers-reduced-motion** setting. If a visitor has asked their device to reduce motion — often for motion sickness or vestibular disorders — all of it switches off automatically. Accessibility was built in, not added afterwards."

The photographs

All 21 product and category photographs, and the hero film, were generated with **Higgsfield** (Nano Banana Pro for stills, Seedance 2.5 for video). They arrived as 4K PNGs totalling about **200 MB**, which would make the site unusable, so a script converts them to WebP at the size they are actually displayed — **2.6 MB for the whole set, a 99% reduction**.

08 The payment system — read this carefully

IMPORTANT

No real money moves, and no card details are ever collected.

The checkout opens a payment sheet that is clearly labelled **"Demo gateway · no real payment"**. It has **no fields for a card number, UPI ID or bank login** — deliberately, so there is nothing to enter and nothing to leak. Choosing a method and pressing pay simply tells our own backend that the order was paid.

Why it is built this way

Taking real payments requires a registered business with a payment provider. This is a college project, so it simulates that step instead. There is also a **"Simulate a failed payment"** button, so the error path can be demonstrated too — not just the happy path.

But the real integration is written

The code for **Razorpay**, India's standard payment gateway, is fully written in `backend/app/services/payment_service.py`: creating a gateway order, and verifying the payment signature using HMAC-SHA256 so a fake confirmation cannot be faked. It stays switched off unless real API keys are configured.

SAY THIS OUT LOUD

"Payments run in mock mode by default. The Razorpay integration is implemented, including HMAC signature verification, but it only activates when API keys are supplied. That way the whole checkout flow can be demonstrated end to end without a merchant account, and without ever handling card data."

Cash on delivery

The second payment option is genuine and needs no gateway at all — the order is confirmed immediately and marked to be paid to the courier.

09 Setting it up on your computer

From nothing to a running website. Do these in order — each step assumes the one before it worked.

Step 0 — install these four things first

INSTALL	GET IT FROM	HOW TO CHECK IT WORKED
Git	git-scm.com	<code>git --version</code>
Node.js (v20+)	nodejs.org	<code>node --version</code>
Python 3.12	python.org	<code>python --version</code>
PostgreSQL 16	postgresql.org	Remember the password you set during install

WATCH OUT

When installing Python on Windows, tick **"Add Python to PATH"** on the very first screen. If you miss it, none of the python commands below will work and you will have to reinstall.

Step 1 — get the code

Open a terminal (on Windows: **Git Bash** or **PowerShell**) and run:

```
# Download the project from GitHub
git clone https://github.com/niri2630/kai-ruchi.git
cd kai-ruchi
```

Step 2 — create the database

```
# Creates an empty database called kairuchi
psql -U postgres -c "CREATE DATABASE kairuchi;"
```

It will ask for the PostgreSQL password you chose during installation.

Step 3 — set up the backend

```
cd backend

# A private Python environment, so this project's packages
# do not clash with anything else on your computer
python -m venv .venv

# Install everything the backend needs
.venv/Scripts/python -m pip install -r requirements.txt

# Copy the settings template
cp .env.example .env
```

Now open backend/.env in any text editor and make sure the database line matches the password you set:

```
DATABASE_URL=postgresql+psycopg2://postgres:YOUR_PASSWORD@localhost:5432/kairuchi
```

Then build the tables and fill in the products:

```
# Create all the tables (this is Alembic doing the migration)
.venv/Scripts/python -m alembic upgrade head

# Load 14 products, 5 categories, 6 demo users and 16 reviews
.venv/Scripts/python -m app.seed
```

Step 4 — set up the frontend

```
cd ../frontend

# Install everything the frontend needs (takes a few minutes)
npm install

# Copy the settings template
cp .env.example .env.local
```

Step 5 — run it

You need **two terminals open at the same time**. This is the part people get wrong: the backend and the frontend are two separate programs.

Terminal 1 — the backend

```
cd backend
.venv/Scripts/python -m uvicorn app.main:app -
-reload --port 8000
```

Leave it running. Check: 127.0.0.1:8000/docs

Terminal 2 — the frontend

```
cd frontend
npm run dev
```

Leave it running. Open: localhost:3000

DEMO LOGIN

Email meera@example.com, password kairuchi123. All six demo accounts use the same password. The sign-in page also has a **"Fill in demo credentials"** button, so you do not have to type it during a demo.

10 When something goes wrong

WHAT YOU SEE	WHAT IT MEANS AND HOW TO FIX IT
"The kitchen isn't answering" on the website	The frontend cannot reach the backend. Almost always: the backend terminal is not running. Start Terminal 1 again.
Products are missing but the site loads	The tables exist but are empty. Run <code>python -m app.seed</code> again.
<code>python: command not found</code>	Python is not on your PATH. Reinstall Python and tick "Add Python to PATH".
<code>password authentication failed</code>	The password in <code>backend/.env</code> does not match your PostgreSQL password.
<code>port 3000 is already in use</code>	The site is already running in another terminal. Close it, or let Next.js use port 3001.
Login stops working after a restart	The <code>SECRET_KEY</code> in <code>.env</code> changed, which invalidates every issued token. Just sign in again.
CORS error in the browser console	The backend is refusing the frontend's address. Add that address to <code>CORS_ORIGINS</code> in <code>backend/.env</code> .

THE FIRST THING TO CHECK, ALWAYS

Open **<http://127.0.0.1:8000/health>**. If it says `"database": "connected"`, the backend and database are both fine and the problem is in the frontend. If it does not load at all, the backend is not running.

11 Questions your teacher will ask

Answer in your own words — but these are the points that matter.

Why did you split the frontend and backend instead of building one program?

So each can be changed, tested and deployed independently, and so the rules live somewhere the user cannot edit. The same backend could serve a mobile app later without rewriting anything.

How are passwords stored?

They are not. Each password is hashed with **bcrypt**, which is one-way — you cannot reverse it. When someone logs in we hash what they typed and compare the hashes. bcrypt also adds a random salt, so two people with the same password get different stored values.

How does the site know I am logged in?

On login the backend issues a **JWT** — a token signed with a secret key. The browser stores it and sends it with every later request. The backend verifies the signature. If anyone edits the token, the signature stops matching and it is rejected.

What is Alembic and why not just create the tables by hand?

Alembic records every schema change as a file. That means the database can be rebuilt identically on any machine with one command, changes can be undone, and the history is in version control alongside the code. Creating tables by hand is not repeatable and cannot be reviewed.

What stops someone changing the price in their browser and paying ₹1?

The browser never sends prices. It sends product IDs and quantities. The backend looks up the real price in the database and recalculates the total itself. It also re-checks stock at that moment, so two people cannot buy the last jar.

Is the payment real?

No, and that is deliberate — a real gateway needs a registered business. The checkout runs a clearly-labelled simulation that collects no card details at all. The Razorpay integration is written, including HMAC signature verification, and activates only if API keys are supplied.

How does order tracking work with no admin panel?

Each status change is saved as a row in `order_events`, and the timeline reads that history. Because there is no staff to click the buttons, a paid order advances on a timer over about 35 minutes so it can be demonstrated live.

How do you know a review is genuine?

When a review is submitted, the backend checks whether that user has an order containing that product. If so it is flagged **verified purchase**. A database constraint also allows only one review per person per product.

Is it responsive?

Yes, and it was tested by measuring the real page at 375 pixels, not by eye. Two real bugs were found and fixed: form fields under 16px caused iOS Safari to zoom the whole page on focus, and the quantity buttons were below the 44-pixel touch target.

What was the hardest part?

A good honest answer: making the cart work for both guests and logged-in users, and merging one into the other at sign-in without losing or duplicating items.

What would you add next?

An admin dashboard so orders and stock can be managed properly, real payments, email notifications, and automated tests.

12 Where everything lives

If you are asked to open a file and show something, this is the map.

```
kai-ruchi/
├── README.md           overview of the whole project
├── DEPLOYMENT.md       how to put it on the internet
├── docs/               this guide
├── backend/            THE PYTHON API
│   ├── alembic/versions/ the migration files (database history)
│   ├── app/
│   │   ├── core/
│   │   │   ├── config.py    all settings, read from .env
│   │   │   ├── database.py  the database connection
│   │   │   ├── security.py  password hashing + JWT tokens
│   │   │   └── deps.py      shared checks (who is logged in, whose cart)
│   │   ├── models/          the 11 database tables as Python classes
│   │   ├── schemas/         validation for data coming in and going out
│   │   ├── routers/         the endpoints – auth, products, cart,
│   │   │                   orders, payments, reviews, contact
│   │   ├── services/        the rules – cart totals, order timeline, payments
│   │   ├── seed.py          the starting catalogue
│   │   └── main.py          where the app starts
│   └── requirements.txt     the Python packages needed
├── frontend/           THE WEBSITE
│   ├── public/
│   │   ├── images/         all 21 photographs (WebP)
│   │   └── videos/          the hero film and category films
│   └── src/
│       ├── app/           one folder per page
│       │   ├── page.tsx    home
│       │   ├── products/    listing + one page per product
│       │   ├── categories/  the five shelves
│       │   ├── cart/ checkout/ basket and payment
│       │   ├── orders/ track/ history and tracking
│       │   ├── login/ signup/ account/
│       │   └── about/ contact/
│       ├── components/
│       │   ├── layout/      navbar, footer, loading screen, smooth scroll
│       │   ├── ui/          buttons, form fields, animations, kolam
│       │   ├── product/     product cards, filters, reviews
│       │   ├── cart/        the slide-out basket
│       │   └── checkout/     the demo payment sheet
│       ├── lib/
│       │   ├── api.ts       every call to the backend, in one file
│       │   └── types.ts     the shape of the data
│       └── store/           cart and login state
```

IF ASKED "SHOW ME THE CODE"

Open `backend/app/routers/cart.py` — it is short, readable, and shows validation, a stock check, an error message and a database write all in one place. Then open `frontend/src/lib/api.ts` to show how the frontend calls it.